

# RideFlow

## Design Document

*Full-Stack Real-Time Ride Booking Platform*

### Deliverable 2

Authors: Tanishq Sharma · Anuj Singh

## 1. Problem Understanding

RideFlow is a real-time ride-booking platform that connects Customers with available Drivers. The system allows customers to request rides, drivers to accept and manage them, and both parties to receive live status updates through WebSockets.

The objective is to provide a lightweight ride-hailing solution demonstrating:

- User Authentication — Separate registration and login for Customers and Drivers
- Ride Booking Workflow — Full lifecycle from request → accept → start → complete
- Driver Assignment — Real-time matching of pending rides with online drivers
- Live Updates — WebSocket-based push notifications for status changes
- Driver Analytics — Performance metrics: total, completed, active rides & rating
- Rating & Feedback — Post-ride rating system with duplicate-prevention logic

## 2. System Architecture

### Frontend

- React.js
- React Router
- Axios
- Tailwind CSS
- STOMP WebSocket Client

### Backend

- Spring Boot
- Spring Web
- Spring Data JPA
- Spring Security
- Spring WebSocket (STOMP)

### Database

- MySQL

### Communication Flow

Customer → REST API → Spring Boot → MySQL

Driver → REST API → Spring Boot → MySQL

Spring Boot → WebSocket (STOMP) → Customer & Driver Dashboards

## Tech Stack Summary

| Layer       | Technology                  | Responsibility                  |
|-------------|-----------------------------|---------------------------------|
| Frontend    | React · Tailwind CSS        | UI, routing, state management   |
| HTTP Client | Axios                       | REST calls to backend           |
| WS Client   | STOMP.js + SockJS           | Real-time event subscription    |
| API Layer   | Spring Boot Controllers     | REST endpoint handling          |
| Security    | Spring Security             | Authentication & access control |
| Business    | Spring Services             | Ride logic, analytics, ratings  |
| Real-Time   | Spring WebSocket            | Broadcasting ride events        |
| ORM         | Spring Data JPA + Hibernate | DB operations & mapping         |
| Database    | MySQL 8                     | Relational data storage         |

## 3. Database Schema

### Customer

| Field       | Type   | Description                 |
|-------------|--------|-----------------------------|
| userId      | Long   | Primary key, auto-generated |
| name        | String | Customer full name          |
| email       | String | Unique login identifier     |
| phoneNumber | String | Contact number              |
| password    | String | Hashed password             |

### Driver

| Field         | Type         | Description                 |
|---------------|--------------|-----------------------------|
| id            | Long         | Primary key, auto-generated |
| name          | String       | Driver full name            |
| phoneNumber   | String       | Contact number              |
| password      | String       | Hashed password             |
| vehicleNumber | String       | Registered vehicle plate    |
| vehicleType   | String       | e.g. Sedan, SUV, Auto       |
| status        | DriverStatus | ONLINE   OFFLINE            |

### Ride

| Field          | Type          | Description                                                |
|----------------|---------------|------------------------------------------------------------|
| rideId         | Long          | Primary key, auto-generated                                |
| customer       | Customer      | FK → Customer (ManyToOne)                                  |
| driver         | Driver        | FK → Driver (ManyToOne, nullable)                          |
| pickupLocation | String        | Pickup address text                                        |
| destination    | String        | Drop-off address text                                      |
| status         | RideStatus    | REQUESTED / ACCEPTED / IN_PROGRESS / COMPLETED / CANCELLED |
| createdAt      | LocalDateTime | Ride request timestamp                                     |

## Rating

| Field    | Type    | Description                 |
|----------|---------|-----------------------------|
| id       | Long    | Primary key, auto-generated |
| ride     | Ride    | FK → Ride (OneToOne)        |
| driver   | Driver  | FK → Driver (ManyToOne)     |
| stars    | Integer | 1-5 star rating             |
| feedback | String  | Optional text feedback      |

## 4. Entity Relationship Diagram (ERD)

Entities and their relationships:

| Relationship    | Cardinality | Description                                     |
|-----------------|-------------|-------------------------------------------------|
| Customer → Ride | 1 : M       | One customer can create many rides              |
| Driver → Ride   | 1 : M       | One driver can be assigned to many rides        |
| Ride → Rating   | 1 : 0..1    | Each completed ride may have at most one rating |

## Ride Status Lifecycle

| Status      | Description                              |
|-------------|------------------------------------------|
| REQUESTED   | Customer books a ride, awaiting a driver |
| ACCEPTED    | A driver has accepted the ride           |
| IN_PROGRESS | Ride is currently active                 |
| COMPLETED   | Ride finished successfully               |
| CANCELLED   | Ride was cancelled at any stage          |

Flow: REQUESTED → ACCEPTED → IN\_PROGRESS → COMPLETED

↓            ↓            ↓  
CANCELLED    CANCELLED    CANCELLED

## 5. API Overview

### Authentication

| Method | Endpoint           | Description                     |
|--------|--------------------|---------------------------------|
| POST   | /register/customer | Register a new customer account |
| POST   | /login/customer    | Authenticate customer           |
| POST   | /register/driver   | Register a new driver account   |
| POST   | /login/driver      | Authenticate driver             |

## Ride Management

| Method | Endpoint                         | Description                        |
|--------|----------------------------------|------------------------------------|
| POST   | /ride/request                    | Customer requests a new ride       |
| GET    | /ride/pending                    | Fetch all rides awaiting a driver  |
| POST   | /ride/{rideId}/accept/{driverId} | Driver accepts a pending ride      |
| POST   | /ride/{rideId}/start             | Driver marks the ride as started   |
| POST   | /ride/{rideId}/complete          | Driver marks the ride as completed |
| POST   | /ride/{rideId}/cancel            | Cancel a ride                      |

## Dashboard & Ratings

| Method | Endpoint                 | Description                     |
|--------|--------------------------|---------------------------------|
| GET    | /customer/{id}/dashboard | Customer analytics              |
| GET    | /driver/{id}/dashboard   | Driver analytics                |
| POST   | /rating/submit           | Submit a star rating + feedback |

## WebSocket

| Endpoint / Topic       | Purpose                                      |
|------------------------|----------------------------------------------|
| ws://localhost:8080/ws | Main WebSocket handshake endpoint            |
| /topic/rides           | Ride status updates to customers and drivers |
| /topic/pending-rides   | New ride broadcasts to all online drivers    |

## 6. Design Decisions

### React + Spring Boot Separation

A decoupled architecture keeps the frontend and backend independently deployable and testable. React's component model fits the multi-dashboard UI, while Spring Boot provides mature REST and WebSocket support with minimal boilerplate.

### MySQL Relational Database

The ride domain has clear relational links (Customer → Ride → Rating, Driver → Ride). A relational database enforces referential integrity and enables JOIN-based analytics queries without denormalization.

### WebSocket via STOMP + SockJS

Polling would waste resources and introduce latency. STOMP topics let the server push events (ride accepted, ride started) instantly to subscribed clients. SockJS provides a fallback for environments that block raw WebSockets.

### **Ride Status State Machine**

Enforcing strict status transitions (REQUESTED → ACCEPTED → IN\_PROGRESS → COMPLETED / CANCELLED) on the backend prevents invalid state mutations and simplifies frontend logic — the UI only needs to react to the current status.

### **Duplicate Rating Prevention**

The Rating entity has a OneToOne relationship with Ride, enforced at the database constraint level. This prevents double-rating without any additional application-layer checks.

### **Driver Online / Offline Toggle**

Drivers are set OFFLINE automatically on logout. This ensures pending-ride broadcasts only reach genuinely available drivers, keeping the pending list clean.